

CS-3510-B Exam 3 (Set A)

Elijah Martin McCauley

TOTAL POINTS

105.5 / 110

QUESTION 1

Problem 1 15 pts

1.1 (i) 3 / 3

- ✓ - 0 pts Correct (True)
- 3 pts Incorrect

1.2 (ii) 3 / 3

- ✓ - 0 pts Correct (False)
- 3 pts Incorrect

1.3 (iii) 3 / 3

- ✓ - 0 pts Correct (False)
- 3 pts Incorrect

1.4 (iv) 3 / 3

- ✓ - 0 pts Correct (False)
- 3 pts Incorrect

1.5 (v) 3 / 3

- ✓ - 0 pts Correct (True)
- 3 pts Incorrect

QUESTION 2

2 Problem 2 5 / 5

- ✓ - 0 pts Correct (d)
- 5 pts Incorrect

QUESTION 3

3 Problem 3 5 / 5

- ✓ - 0 pts Correct (d)
- 5 pts Incorrect

QUESTION 4

4 Problem 4 25 / 25

- ✓ - 0 pts Correct

Table Definition (5 pts)

- 0 pts Dimensions missing (No Penalty)
- 2 pts Minor mistake

(Ex. Inclusion of the i^{th} element not specified)

- 3 pts Mostly incorrect table definition
- 5 pts Table Definition Missing

Base Cases and Recurrence (20 pts)

- 2 pts Minor error in base case

(Ex. $T[1] \neq 0$, More than one provided including the correct one, but not required for the recursion)

- 4 pts Major error in base case

(Ex. Base Case not supporting the recursion)

- 5 pts Base Case Missing
- 3 pts Minor mistake in recurrence
- 5 pts Recurrence does not align with the table defined.

- 7 pts Major Error in recurrence/ Inefficient solution (higher than $O(n^2)$)

- **10 pts** Recurrence Missing
- **2 pts** Minor error in the explanation
- **4 pts** Major error in the explanation
- **5 pts** Explanation of correctness Missing
- **25 pts** Missing Answer

QUESTION 5

5 Problem 5 25 / 25

✓ - **0 pts** Correct

Table Definition (5 pts)

- **0 pts** Dimensions missing (No Penalty)
- **2 pts** Minor mistake

(Ex. Inclusion of the i^{th} element not specified)

- **3 pts** Mostly incorrect table definition
- **5 pts** Table Definition Missing

Base Cases and Recurrence (20 pts)

- **2 pts** Minor error in base case

(Ex. More base cases provided including the correct one, but not required for the recursion)

- **4 pts** Major error in base case

(Ex. Base Case not supporting the recursion)

- **5 pts** Base Case Missing
- **3 pts** Minor mistake in recurrence
- **5 pts** Recurrence does not align with the table defined.

- **7 pts** Major Error in recurrence/ Inefficient solution (higher than $O(n^2)$)

- **10 pts** Recurrence Missing
- **2 pts** Minor error in the explanation
- **4 pts** Major error in the explanation
- **5 pts** Explanation of correctness Missing
- **25 pts** Missing Answer

QUESTION 6

6 Problem 6 25 / 25

✓ - **0 pts** Correct

Table Definition (5 pts)

- **0 pts** Dimensions missing (No Penalty)
- **2 pts** Minor mistake

(Ex. Inclusion of the i^{th} element not specified)

- **3 pts** Mostly incorrect table definition
- **5 pts** Table Definition Missing

Base Cases and Recurrence (20 pts)

- **2 pts** Minor error in base case

(Ex. More base cases provided including the correct one, but not required for the recursion)

- **4 pts** Major error in base case

(Ex. Base Case not supporting the recursion)

- **5 pts** Base Case Missing
- **3 pts** Minor mistake in recurrence
- **5 pts** Recurrence does not align with the table defined.

- **7 pts** Major Error in recurrence/ Inefficient solution (higher than $O(n^2)$)

- **10 pts** Recurrence Missing
- **2 pts** Minor error in the explanation
- **4 pts** Major error in the explanation
- **5 pts** Explanation of correctness Missing
- **25 pts** Missing Answer

QUESTION 7

7 Problem 7 (Extra Credit) 5.5 / 10

- **0 pts** Correct

Table Definition (1 pts)

- **0 pts** Correct/ Dimensions missing (No Penalty)

✓ - **0.5 pts** *Incorrect, but the recurrence is consistent with table definition*

- **1 pts** Incorrect w/ no recurrence

Base Cases and Recurrence (9 pts)

- **1 pts** Minor error in base case

(Ex. More base cases provided including the correct one, but not required for the recursion)

- **2 pts** Missing/ Major error in base case

(Ex. Base Case not supporting the recursion)

- **2 pts** Minor mistake in recurrence

- **4 pts** Recurrence does not align with the table defined.

✓ - **4 pts** *Major Error in recurrence/ Inefficient solution (higher than $O(n^2)$ or $O(n \log n)$)*

- **5 pts** Recurrence Missing

- **2 pts** Missing/ Incorrect Explanation of correctness

- **1 pts** Wrong return value

- **10 pts** Missing Answer

GTID: 903684653
NAME: Elijah McCauley

Georgia Institute of Technology

Spring 2023

CS 3510B – Design & Analysis of Algorithms
Exam 3 (Set A)

March 16, 2023

TIME ALLOWED: 75 MINS

Name: Elijah McCauley
GTID: 903684653
GT Username: emccauley6

INSTRUCTIONS TO CANDIDATES

1. Please write your NAME and GTID clearly on all the pages.
2. This examination paper is worth **110 points** (100 + 10 extra credit points) comprising **SEVEN (7)** questions and **TWELVE** printed pages.
3. **ONLY** write on the front sheets of paper that are numbered. The backs will not be scanned.
4. Wherever a blank is provided, please write your answer in the given blank.
5. Calculators are **NOT** allowed.

I am in aware of and the accordance with Academic Honor Code of Georgia Tech and the Georgia Tech Code of Conduct. I'll use no external help on this test. Also, I have read all the instructions on this page.

Signature: 

Problem 1 (15 points; 3 points each)

Indicate whether the following statements are **true** or **false**.

- (i) In the Longest Common Subsequence problem, $T[i, j]$ represents the length of the longest common subsequence of $X[1 \dots i]$, $Y[1 \dots j]$.

Answer: True

- (ii) The value we return in a dynamic programming algorithm is always a single entry in the DP table, like $T[i]$ or $T[i, j]$.

Answer: False

- (iii) Suppose we are given n intervals (l_i, r_i) for $i = 1, \dots, n$ and we would like to find a set S of non-overlapping intervals maximizing $\sum_{i \in S} w_i$, where w_i represents the weight of interval (l_i, r_i) .

Consider the following greedy algorithm: Select (in the set S) the interval, say (l_i, r_i) of maximum weight w_i , remove all intervals that overlap with (l_i, r_i) , and repeat. This algorithm always provides an optimum solution to this interval selection problem.

Answer: False

- (iv) Running Bellman-Ford on a graph G doesn't help detect a negative weight cycle, since the shortest path for a graph with negative weight cycle is undefined.

Answer: False

- (v) In the Bellman-Ford algorithm on a graph with no negative weighted cycle, the maximum number of times that the distance label of a given node i can be reduced is equal to $n - 1$, where n is the number of nodes in the graph.

Answer: True

Problem 2 (5 points)

A "boundedly changing" sequence is one in which the absolute difference between consecutive elements is at least d . For $d = 2$, the sequence 1, 4, 2, 11, 8, 7 is not a boundedly changing sequence and 4, 7, 5, 11, 9, 6 is a boundedly changing sequence.

Let A be a length n sequence of integers. Suppose you want to design a dynamic programming algorithm to find the length of the longest boundedly changing subsequence of A . While there may be many solutions to this problem, one of the recurrences on the list below would achieve a working algorithm.

Select the correct recurrence relation. You do not need to explain your answer.

Write the correct letter option (a, b, c, ...) in the given blank: d

- a. $T[i] = \max\{T[j] : 1 \leq j < i, A_j - A_i \leq d\} + 1$
- b. $T[i] = \max\{T[j] : 1 \leq j < i, A_j - A_i \geq d\} + 1$
- c. $T[i] = \max\{T[i - A_j] : 1 \leq j < i, |A_j - A_i| \geq d\} + 1$
- d. $T[i] = \max\{T[j] : 1 \leq j < i, |A_j - A_i| \geq d\} + 1$
- e. $T[i] = \max\{T[j] : 1 \leq j < i, A_j < A_i\} + d$
- f. $T[i] = \max\{T[i - 1], A_i + d\}$
- g. $T[i] = \max\{T[i - d], A_i + 1\}$

$$T[i] = \max \{$$

1 4 2 11 8 7
 1 2

$$|A_j - A_i| \geq d$$

$$|1 - 4| \geq 2$$

$$2 - 4 \geq 2$$

Problem 3 (5 points)

You are given a target value V and an array of coin denominations $D = [d_1, d_2, \dots, d_n]$. You need to make a change for V using the smallest number of coins. The coins have some bounded number of denominations, but an unbounded number of available coins for each denomination i.e. you can use each coin for an infinite amount of time. Suppose you want to design a dynamic programming algorithm to find change for a given amount V using the minimum number of coins from a set of denominations assuming that $T[n][V - 1]$ stores the minimum number of coins required. Select the correct recurrence relation. You do not need to explain your answer.

$\wedge = \text{num coins}$

Examples:

Input: $V = 8$, $D = [1, 4, 6]$ Note: You have an infinite number of each coin.

Output: 2; *Explanation:* You can use two coins of denomination 4, and that adds up to the value 8.

Write the correct letter option (a, b, c, ...) in the given blank: d

a.

$$T[i][j] = \begin{cases} T[i][j - 1], & \text{if } j < d_i \\ \min(T[i, j - 1], 1 + T[i][j - d_i]), & \text{else} \end{cases}$$

b.

$$T[i][j] = \begin{cases} T[i - 1][j], & \text{if } j < d_i \\ \min(T[i - 1][j], 1 + T[i - d_i][j]), & \text{else} \end{cases}$$

c.

$$T[i][j] = \begin{cases} T[i][j - 1], & \text{if } j < d_i \\ \min(T[i][j - 1], 1 + T[i][j - d_i]), & \text{else} \end{cases}$$

d.

$$T[i][j] = \begin{cases} T[i - 1][j], & \text{if } j < d_i \\ \min(T[i - 1][j], 1 + T[i][j - d_i]), & \text{else} \end{cases}$$

e.

$$T[i][j] = \begin{cases} T[i - 1][j], & \text{if } j < d_i \\ 1 + T[i][j - d_i], & \text{else} \end{cases}$$

Problem 4 (25 points)

Imagine you are practicing for the long jump by playing a game as a member of the Georgia Tech Athletics team. You are given an array of squares $S = [s_1, s_2, \dots, s_n]$ where $s_i \leq n$. You start at position s_1 on a line of squares, and with each turn, you can jump some number of squares forward. However, you are only allowed to jump a distance *up to* the number written on your current square. For example, if the number of your square is 5, then you could jump either 1, 2, 3, 4, or 5 squares ahead.

Design a dynamic programming algorithm to find the minimum number of jumps (so the number on the final square is useless because you can't jump past square n) to jump to the final square s_n in S .

Input: $S = [s_1, s_2, \dots, s_n] = [1, 4, 2, 1, 3, 0, 8, 4]$

Output: 3

Explanation: You can jump from s_1 with a value of 1 to s_2 . From s_2 , with a value of 4, you can jump 3 squares (since with a value of 4 you can jump 1, 2, 3, or 4 squares) to s_5 with a value of 3. From s_5 you can jump 3 squares to $s_8 = s_n$. However, you can jump 4 squares from s_2 , but doing so will bring you to s_6 with a value of 0. Hence, you can't move from s_6 .

(i) (5 points) Define the entries of your table in words. E.g. $T[i]$ or $T[i][j]$ is ...

$T[i]$ = minimum number of jumps needed to reach i . We would return the value in $T[n]$ as that would be the minimum jumps to reach the final square.

- (ii) (20 points) State the base cases and recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.
Full points will be awarded to the most efficient solutions.

The base case is $T[1]$ (assuming our array starts at index 1) = 0 because it takes no jumps to get to the starting point in the array.

Our recurrence relation is

$$T[i] = \left(\underset{1 \leq j < i}{\text{minimum}} (T[j] + 1) \right) \text{ when } s_j \geq i - j$$

loop through
 $1 \leq j < i$
and take the
minimum
of $T[j] + 1$
but only
for
when
 $s_j \geq i - j$

I know this isn't the clearest way to write it but basically it's the minimum of $T[j] + 1$ for some $1 \leq j < i$ but only if $s_j \geq i - j$ because that means that you can reach i from j .

This is right because if $s_j < i - j$ you cannot reach i from j so you can't count it but also you want the min because multiple j can probably reach i .

Problem 5 (25 points)

You are given two positive integer arrays called *array1* and *array2*. The longest opposing subsequences are the longest equal-length subsequences S of *array1* and T of *array2* such that $S[i] \not\equiv T[i] \pmod{2}$ for all i . In other words, $S[i]$ and $T[i]$ CANNOT be both even or both odd for any index i .

Design a dynamic programming algorithm to find the length of the longest opposing subsequences between *array1* and *array2*.

Input: *array1* = [5, 7, 6, 4, 1, 8, 9], *array2* = [3, 10, 7, 9, 4]

Output: 4

Explanation: $S = [5, 6, 8, 9]$ and $T = [10, 7, 9, 4]$. There are multiple opposing sequences of length 4. But, $[7, 6, 4, 8]$ and $[10, 7, 9, 4]$ is *not* the longest opposing subsequence because elements at $i = 4^{\text{th}}$ in *array1* and *array2* are both even.

(i) (5 points) Define the entries of your table in words. E.g. $T[i]$ or $T[i][j]$ is ...

$T[i, j]$ where $T[i, j] = \text{length of the longest opposing subsequence of } S[1 \dots i] \text{ and } V[1 \dots j]$

- (ii) (20 points) State the base cases and recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.
Full points will be awarded to the most efficient solutions.

The base case is the first row and col $T[0, j]$ and $T[i, 0]$ which both = 0.

The recurrence relation is

$$T[i, j] = \begin{cases} \max(T[i-1, j], T[i, j-1]) & \text{if } S[i] \bmod 2 == V[j] \bmod 2 \\ T[i-1, j-1] + 1 & \text{if } S[i] \bmod 2 \neq V[j] \bmod 2 \end{cases}$$

This is extremely similar to LCS, but the difference is in the conditional.

In this case, we want to take the previous longest opposing subsequence, which would have to be in $T[i-1, j-1]$ and add 1 to it to get the new longest opposing subsequence. If $S[i] \bmod 2$ and $V[j] \bmod 2$ match then we take the longest opposing subsequence from $S[1 \dots i-1], V[1 \dots j]$ or

We would return the entry in

$T[S.length, V.length]$

⁸ $S[1 \dots i], V[1 \dots j-1]$ which are located at $T[i-1, j]$ and $T[i, j-1]$ respectively.

Problem 6 (25 points)

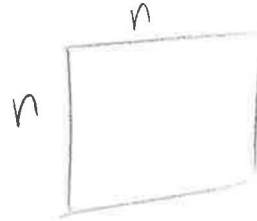
You are given a Georgia Tech map in the form of a $n \times n$ matrix M of non-negative integers. Suppose you walk along the matrix starting at coordinate $(1, 1)$ in order to reach coordinate (n, n) accumulating the integers along the way. At any coordinate, you may only move right or down. That is, if you are at coordinate (i, j) , you can only move to either $(i + 1, j)$ or $(i, j + 1)$ in a single step.

Design a dynamic programming algorithm to find the **minimum** sum you can accumulate in your walk subject to the above constraints.

Example

Input:

$$M = \begin{bmatrix} 2 & 1 & 4 \\ 1 & 3 & 5 \\ 9 & 0 & 1 \end{bmatrix}$$



Output: 7

Explanation: This sum comes from the path $(1, 1) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (3, 2) \rightarrow (3, 3)$. The accumulated sum is $M[1][1] + M[1][2] + M[2][2] + M[3][2] + M[3][3] = 2 + 1 + 3 + 0 + 1 = 7$.

(i) (5 points) Define the entries of your table in words. E.g. $T[i]$ or $T[i][j]$ is ...

$T[i, j]$ = minimum sum you can achieve while getting to $M[i][j]$.

so $T[1, 1]$ is the minimum sum to get to the start while

$T[n, n]$ is the minimum sum to get to the last square.

feels like bellman

take at most 8 steps

Problem 7 (Extra Credit; 10 points)

In this problem, you have two parallel lines and two arrays A and B consisting of n elements each. These values represent distinct points on the two lines. The points given in array A are sorted in increasing order, but the points in array B can be in any order. The goal is to create as many bridges between the two lines following these constraints:

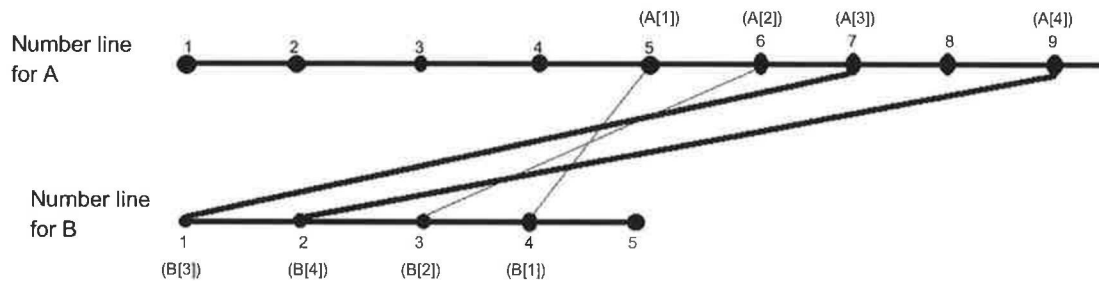
- (a) A bridge can only be made between coordinates $A[i]$ and $B[i]$ for $1 \leq i \leq n$.
- (b) For all pairs of bridges $(A[i], B[i])$ and $(A[j], B[j])$ the bridges can't 'cross' each other. In other words, if $A[i] < A[j]$ then you can't have $B[i] > B[j]$ (sketch it out and you'll realize bridge i and bridge j will intersect at some point). Similarly, if $A[i] > A[j]$ then you can't have $B[i] < B[j]$.

Design a dynamic programming algorithm as you can to calculate the maximum number of bridges that can be constructed

Input: $A = [5, 6, 7, 9]$, $B = [4, 3, 1, 2]$, $n = 4$

Output: 2

Explanation: The coordinates in A are already sorted, so to make sure we don't have crossing bridges, we need to select locations such that if $A[i] < A[j]$ then $B[i] < B[j]$ (where i and j are the indices representing chosen locations). In this instance, any sequence of length 3 will violate the condition. For instance, consider indices 2, 3, 4. $A[2] < A[3]$ but we can see that $B[2] = 3 > B[3] = 1$ so if we construct bridges with both of these locations they will 'cross' each other. The optimal locations for this example would be indices [3, 4] (so locations $(A[3] = 7, B[3] = 1)$ and $(A[4] = 9, B[4] = 2)$)



- (i) (1 point) Define the entries of your table in words. E.g. $T[i]$ or $T[i][j]$ is ...

$T[i, j]$ is max number of bridges
 using $A[1...i]$ and
 $B[1...j]$

- (ii) (9 points) State the base cases and recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Full points will be awarded to the most efficient solutions.

Base case $T[1, j] = 1$ because if
 $T[i, 1] = 1$ one of the
 arrays
 has only one
 entry, there
 cannot be a conflict

Recurrence

$$T[i, j] = \begin{cases} \max(T[i-1, j], T[i, j-1]) & \text{if } A[i] < A[j] \\ & \text{and } B[i] > B[j] \\ T[i-1, j-1] + 1 & \text{or } A[i] > A[j] \\ & \text{and } B[i] < B[j] \end{cases}$$